

✳ 第一阶段：视觉语言特征对齐（Pre-training for Feature Alignment）

这一阶段可以形象地称为“语言学校的速成班”。

1. 深度剖析：它在解决什么问题？

在训练开始前，大模型（Vicuna）是个纯文科生，只认识文本词向量（Text Tokens）；视觉编码器（CLIP）是个画家，它输出的是图像特征向量（Vision Patches）。虽然它们各自在自己的领域很强，但它们的向量空间完全不互通。如果直接把 CLIP 提取出来的向量扔给 Vicuna，Vicuna 看到的就是一堆无意义的乱码。

所以，第一阶段的本质就是：找一个平价的“翻译官”（Linear Projection 矩阵 W ），只训练它，让它把视觉向量翻译成大模型能听懂的“外语”。

2. 数据集的秘密：CC-3M 的子集（约 59 万条）

这一阶段用的数据非常简单，形式通常是：

- 图片：一张猫的图片。
- 文本：“Describe the image briefly.” -> “A fluffy orange cat sitting on a sofa.”（简要描述这张图。-> 一只蓬松的橘猫坐在沙发上。）

这些数据没有复杂的推理，只有纯粹的“图片 ↔ 标签/短句”的对应关系。

3. 数据是如何流动的？（前向传播细节）

- Step 1: 图片输入给 CLIP，CLIP 将其切片并转化为特征向量 X_v 。
- Step 2: X_v 乘以投影矩阵 W ，得到 $H_v = W \cdot X_v$ 。此时， H_v 的维度已经和 Vicuna 的文本 Token 维度完全一致了。
- Step 3: 把 H_v 和提示词 “Describe the image briefly.” 的文本 Token 拼在一起，作为一个长序列扔给 Vicuna。
- Step 4: Vicuna 吐出预测文本。计算它吐出的文本与标准答案（“A fluffy orange cat...”）之间的交叉熵损失（Cross-Entropy Loss）。
- Step 5: 关键反向传播！只有矩阵 W 的梯度被计算并更新。Vicuna 和 CLIP 的参数就像被水泥浇筑了一样，一动不动。

转维度。



图片 & 简单文本

固定难训练

4. 为什么这么设计？（Why）

- 保护大模型的大脑：Vicuna 本身已经具备了极强的语言理解和生成能力。如果在数据这么简单的阶段就动它的参数，容易让它“变傻”（发生灾难性遗忘）。
- 效率极高：只训练一个简单的线性层（单层矩阵乘以向量），显存占用极小，训练速度飞快。

第二阶段：端到端微调（End-to-end Fine-tuning）

这一阶段可以称为“多模态特种兵进阶训练”。

1. 深度剖析：它在解决什么问题？

经过第一阶段，Vicuna 已经勉强能看懂图片里有啥了，但它现在只会做低级的“看图说话”。如果你问它：“图里这个人为什么笑了？背后的社会学原因是什么？”或者“根据图中的电路图，下一步应该接哪根线？”，它直接就懵了。

因为第一阶段训练出的“翻译官”能力有限，它只能把视觉特征硬塞进文本空间，并没有让**图像和复杂的逻辑推理、上下文对话深度融合**。第二阶段就是要打破藩篱，**让大模型的大脑亲自参与图像的理解与思考**。

2. 数据集的巧思：15 万条 GPT-4 指令数据

就是上一轮提到的，包含**多轮对话、详细描述、复杂推理**的高级数据。

3. 数据是如何流动的？（前向传播细节）

- Step 1 & 2: 图片依然通过 CLIP，并通过已经具备翻译能力的投影层 W ，转化为 H_v 。
- Step 3: 人类输入复杂的指令（例如：“如果你是图中的店主，你会怎么重新陈列这些商品以提高销量？请给出三点建议。”）。
- Step 4: H_v 与复杂指令拼接，送入 Vicuna。
- Step 5: 关键反向传播！计算损失后，**梯度同时回传给“翻译官” W 和“大脑” Vicuna。**
 - W 会进一步微调，翻译得更传神。
 - Vicuna 的内部权重发生改变，学会了**如何把视觉信息融入到自己的知识网络中，进行逻辑推理和长文本输出**。
 - 注：**CLIP 在这里依然是冻结的**。

4. 为什么这么设计？（Why）

- 释放真正的多模态智能：只有解冻 LLM，模型才能学会“根据视觉线索调整推理逻辑”。
- 为什么依然冻结视觉编码器（CLIP）？
 - CLIP 是在几亿张图片上预训练好的，视觉感知能力已经见顶了，再动它性价比不高。
 - 如果连 CLIP 一起解冻，显存开销会呈指数级暴涨，普通实验室根本训不动。

以下是核心代码：

🔗 重点核心一：LlavaMultiModalProjector 类（多模态翻译官）

寻找代码第 71 行。这就是连接视觉和文本的“线性投影层”(MLP)，它的核心就在于 `__init__` 里的网络定义：

Python

```
self.linear_1 = nn.Linear(
    config.vision_config.hidden_size * num_feature_layers,
    config.text_config.hidden_size,
    bias=config.multimodal_projector_bias,
)
self.act = ACT2FN[config.projector_hidden_act]
self.linear_2 = nn.Linear(
    config.text_config.hidden_size, config.text_config.hidden_size, bias=config.multimodal
```

💡 核心考点：

- **结构**：它是由两个 `nn.Linear` 夹着一个激活函数（通常是 `GELU`）组成的 **两层 MLP**（LLaVA 1.0 是单层线性层，LLaVA 1.5 升级为了两层 MLP，效果大幅提升）。
- **维度转换**：它的本质是将视觉特征的通道数（比如 CLIP 输出的 1024 维），映射成大语言模型需要的通道数（比如 LLaMA/Vicuna 认的 4096 维）。

🔗 重点核心二：LlavaModel 中的 `get_image_features` 函数

寻找代码第 119 行。这个函数负责提取并加工视觉特征：

Python

```
image_outputs = self.vision_tower(pixel_values, output_hidden_states=True, ...)

if isinstance(vision_feature_layer, int):
    selected_image_feature = image_outputs.hidden_states[vision_feature_layer]
    if vision_feature_select_strategy == "default":
        selected_image_feature = selected_image_feature[:, 1:] # 剔除 CLS Token
else:
    # 或者是把多层的特征拿出来进行 torch.cat(..., dim=-1) 拼接
```

💡 核心考点：

- **特征选择策略**：为什么要用 `[:, 1:]`？因为 CLIP 提取特征时，输出的第一个 Token 是用于图像分类的 [CLS] Token，后面的才是代表图像局部区域（Patches）的局部特征。LLaVA 不需要分类，它需要局部细节，所以把第一个 [CLS] Token 扔掉，只留后面的空间特征。
- **多层特征融合**：代码中支持 `isinstance(vision_feature_layer, list)`。LLaVA 1.5 发现光拿 CLIP 最后一层特征不够丰富，所以允许把倒数第二层、第三层的特征在最后一个维度（`dim=-1`）上拼起来（`torch.cat`），然后再送进 Projector 翻译。

（但是纵深的）
（来自于不同的块）
→ 全方位，多视角的细节特征

🔥 重点核心三： `LlavaModel.forward` 里的“融合魔术”(最重要!)

寻找代码第 185 行。这是整个多模态大模型最精彩、最精妙的部分。请死死盯住 `forward` 里的这三行代码：

Python

```
if pixel_values is not None:
    # 1. 拿到经过 MLP 翻译后的图像特征矩阵
    image_features = self.get_image_features(...)

    # 2. 找到文本中特殊的 <image> Token 标记在哪个位置, 生成一个掩码矩阵
    special_image_mask = self.get_placeholder_mask(...)

    # 3. 施展魔术: 用图像特征, 强行替换掉文本 Embedding 中 <image> 的位置
    inputs_embeds = inputs_embeds.masked_scatter(special_image_mask, image_features)
```

32000
↑
[[[[[...]]], [[...]], [[...]]]
这张图像是一只——

💡 核心考点：

- `masked_scatter` 的妙用：在文本被 `Tokenizer` 编码时，遇到图片，文本里会留下一个占位符 `"<image>"`（比如其 `Token ID` 是 32000）。通过 `masked_scatter` 函数，代码用真正的图像特征（Tensor）去精准覆盖并替代掉文本矩阵里那些代表 `"<image>"` 的坑位。
- 这样一来，原先纯文本的 `inputs_embeds` 在经过这行代码后，就变成了“图文交织”的混合 `Embedding` 阵列。紧接着代码调用 `self.language_model(inputs_embeds=inputs_embeds)`，直接就把这个混合阵列像喂纯文本一样喂给了大语言模型。大模型在毫不知情的情况下，就把图像当作几个特殊的“神秘单词”一起处理了！

🔥 重点核心四： `prepare_inputs_for_generation` (KV Cache 优化)

寻找代码第 286 行。在模型生成对话（推理）时，这个函数起到了决定性的加速作用：

Python

```
if is_first_iteration or not kwargs.get("use_cache", True):
    model_inputs["pixel_values"] = pixel_values
```

💡 核心考点：

- 为什么只在第一轮（`is_first_iteration`）传 `pixel_values`？因为图片非常大，经过 CLIP 切片后会变成几百个图像 Tokens。在第一轮 `Prefill`（预填充）阶段，图片已经被送进去转成特征，并作为大模型的 `KV Cache`（键值缓存）存起来了。
- 在随后的自回归生成阶段（预测第 2 个字、第 3 个字时），模型只需要关注新生成的字，不需要重复处理图片。因此后续迭代将 `pixel_values` 设为 `None`，极大地省去了重复计算视觉编码器的开销。

1轮：张图 → KV Cache。
以后！不传图，
关键新生成字 + 记忆去写